



LABORATORIO 2

MÉTODOS DE ORDENAMIENTO Y BÚSQUEDA

Servidor Java y comunicación HTTP

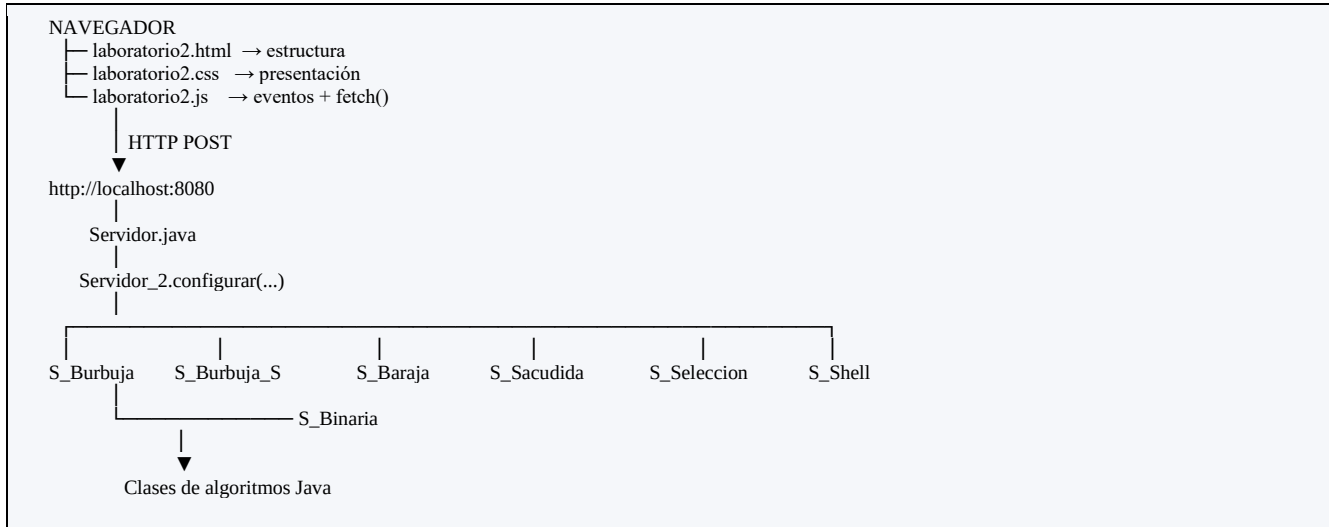
Contenido

- 1 Introducción
- 2 Arquitectura general
- 3 Servidor principal
- 4 Servidor_2
- 5 Clases S_ del Laboratorio 2
- 6 Endpoints
- 7 CORS y OPTIONS
- 8 Recepción de formularios
- 9 Respuestas JSON
- 10 Ordenamiento desde la web
- 11 Búsqueda binaria desde la web
- 12 Manejo de errores
- 13 Flujo completo
- 14 Conclusiones

1. Introducción

El servidor Java funciona como puente entre la página web y las clases que implementan los algoritmos. La página se ejecuta en el navegador, mientras Java escucha solicitudes HTTP en el puerto 8080. El frontend no reemplaza los algoritmos: envía los datos, Java procesa y devuelve la respuesta.

2. Arquitectura general



3. Servidor principal

El proyecto utiliza un único `HttpServer`. El servidor principal crea la instancia en el puerto 8080, registra la ruta raíz y delega el registro de las rutas del Laboratorio 2. Esta decisión evita levantar un servidor distinto para cada laboratorio.

```
HttpServer servidor = HttpServer.create(
    new InetSocketAddress(8080),
    0
);

// Configuración del Laboratorio 2
Servidor_2.configurar(servidor);

servidor.start();
```

Idea clave: Las clases `S_` no deben crear otro `HttpServer` ni tener un `main` independiente. Reciben el servidor ya creado y registran su contexto.

4. Servidor_2

`Servidor_2` actúa como módulo de rutas del Laboratorio 2. Su responsabilidad es registrar la ruta de comprobación `/lab2` y llamar a `configurar(...)` de las clases servidoras específicas.

```
public static void configurar(HttpServer servidor) {
    servidor.createContext("/lab2", exchange -> {
        agregarCORS(exchange);
        // respuesta de comprobación del Laboratorio 2
    });

    S_Burbuja.configurar(servidor);
}
```

```

S_Burbuja.S.configurar(servidor);
S_Baraja.configurar(servidor);
S_Sacudida.configurar(servidor);
S_Seleccion.configurar(servidor);
S_Shell.configurar(servidor);
S_Binaria.configurar(servidor);
}

```

5. Clases S_ del Laboratorio 2

Cada clase S_ representa la capa HTTP de un algoritmo. Su función es recibir la petición, validar el método HTTP, extraer tam y elementos, invocar la clase de lógica y devolver JSON. El patrón se repite para los seis métodos; cambia la ruta y la clase de algoritmo.

Clase servidor	Responsabilidad	Lógica asociada
S_Burbuja	Atender /lab2/burbuja	Burbuja
S_Burbuja_S	Atender /lab2/burbuja-senal	Burbuja con señal
S_Baraja	Atender /lab2/baraja	Baraja
S_Sacudida	Atender /lab2/sacudida	Sacudida
S_Seleccion	Atender /lab2/seleccion	Selección
S_Shell	Atender /lab2/shell	Shell
S_Binaria	Atender /lab2/binaria	Búsqueda binaria

6. Endpoints

El frontend actual configura explícitamente las rutas de los seis métodos y utiliza /lab2/binaria para búsqueda.

Ruta	HTTP	Datos principales	Resultado esperado
/lab2/burbuja	POST	tam, elementos	original, ordenado
/lab2/burbuja-senal	POST	tam, elementos	original, ordenado
/lab2/baraja	POST	tam, elementos	original, ordenado
/lab2/sacudida	POST	tam, elementos	original, ordenado
/lab2/seleccion	POST	tam, elementos	original, ordenado
/lab2/shell	POST	tam, elementos	original, ordenado

/lab2/binaria	POST	tam, elementos, metodo, tipo, valor	arreglos + datos de búsqueda
---------------	------	-------------------------------------	------------------------------

7. CORS y OPTIONS

Como la página puede ejecutarse con Live Server en un origen distinto a localhost:8080, el servidor agrega encabezados CORS. También debe responder a OPTIONS cuando el navegador realiza una solicitud previa.

```
exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*");
exchange.getResponseHeaders().add("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
exchange.getResponseHeaders().add("Access-Control-Allow-Headers", "Content-Type");

if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
    exchange.sendResponseHeaders(204, -1);
    exchange.close();
    return;
}
```

8. Recepción de formularios

JavaScript envía los campos como application/x-www-form-urlencoded. Java lee el cuerpo, lo decodifica en UTF-8 y obtiene pares nombre=valor. En ordenamiento los campos esenciales son tam y elementos.

```
String cuerpo = new String(
    exchange.getRequestBody().readAllBytes(),
    StandardCharsets.UTF_8
);
// Ejemplo:
// tam=5&elementos=8%2C3%2C7%2C1%2C5
```

9. Respuestas JSON

La respuesta permite que JavaScript separe estado, mensaje, arreglo original y arreglo ordenado. Un ejemplo conceptual es:

```
{
  "ok": true,
  "mensaje": "Arreglo ordenado correctamente",
  "original": [8, 3, 7, 1, 5],
  "ordenado": [1, 3, 5, 7, 8]
}
```

Para búsqueda se agregan campos como tipoBusqueda, encontrado, indiceDesordenado, indiceOrdenado, numeroDesordenado o numeroOrdenado, según el modo utilizado.

10. Ordenamiento desde la web

El navegador selecciona la ruta según el método. JavaScript envía tam y elementos por POST; el servidor valida, ejecuta el algoritmo y devuelve ambos arreglos. Después showArrays(...) los representa visualmente.

```
const data = await postForm(
  methodConfig.route,
  {
    tam: arrayInput.tam,
    elementos: arrayInput.elementos
  }
);
```

```
    },  
    controller.signal  
  );  
  
  validateServerArrays(data);  
  showArrays(data.original, data.ordenado);
```

11. Búsqueda binaria desde la web

La búsqueda usa una única ruta `/lab2/binaria`. Además del arreglo se envían el método de ordenamiento seleccionado, el tipo de búsqueda y el valor. Esto permite ordenar primero y después ejecutar la búsqueda sobre el arreglo ordenado.

```
const data = await postForm(  
  "/lab2/binaria",  
  {  
    tam: arrayInput.tam,  
    elementos: arrayInput.elementos,  
    metodo: methodConfig.backendValue,  
    tipo: SEARCH_TYPE_CONFIG[searchMode],  
    valor: searchInput.value  
  },  
  controller.signal  
);
```

Búsqueda por número

Devuelve la posición del número en el arreglo original y en el ordenado. Si no existe, el servidor devuelve un estado/mensaje de no encontrado.

Búsqueda por índice

El índice se interpreta como una posición. La interfaz muestra el valor que ocupa esa posición en el arreglo original y el valor de la misma posición en el arreglo ordenado.

12. Manejo de errores

- 405: método HTTP no permitido.
- Errores de validación: tamaño inválido, cantidad incorrecta, números no enteros o repetidos.
- Error de conexión: el frontend informa cuando Java no está ejecutándose en localhost:8080.
- Respuesta no válida: JavaScript intenta convertir la respuesta a JSON y muestra el mensaje recibido si falla.
- AbortController: permite cancelar una petición activa cuando el usuario modifica el arreglo o cierra una operación.

13. Flujo completo

1. El usuario abre laboratorio2.html.
2. Confirma el tamaño del arreglo.
3. Ingresa exactamente la cantidad indicada de números enteros.
4. Selecciona uno de los seis métodos.
5. Al pulsar Ordenar, JavaScript realiza POST al endpoint del método.
6. Java procesa el arreglo y responde JSON.
7. JavaScript muestra Comparación de arreglos.
8. Al pulsar Buscar se abre Búsqueda binaria.
9. JavaScript envía el arreglo, método, tipo y valor a `/lab2/binaria`.
10. Java devuelve los datos y la interfaz presenta el resultado.

14. Conclusiones

La modularización separa responsabilidades: el servidor principal administra el puerto; Servidor_2 agrupa el Laboratorio 2; las clases S_ traducen HTTP a llamadas Java; las clases de algoritmos realizan el procesamiento; y JavaScript se encarga de la interacción visual. Esta estructura facilita añadir laboratorios sin convertir Servidor.java en un archivo monolítico.